两个性能升级的业务承诺

ETag 卸 CDN 流量 & 削峰填谷救双 11 下单漏斗

gomall · 业务侧 deck

今日大纲

业务困局：两个不是” 新功能” 的事

HTTP cache 的业务承诺

异步下单的业务承诺

压测数字对照

回到业务：性能即业务

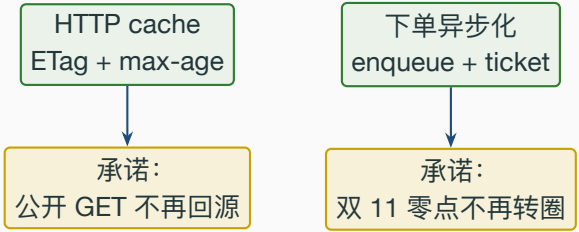
业务困局：两个不是” 新功能” 的事

这不是加功能，是”挽救已有功能”

- 商品详情这个接口已经写了 N 个版本：DB → Redis → ES。每加一层都为了”再快一点 / 再省一点”。
- `/orders/create` 也跑了 3 年了，平时 30–50ms 一笔，没人抱怨。
- 但有两件事每年至少坏一次：
  - **运营推爆款**：100 万人同时刷一个商品 → Redis 命中率掉到 71%，DB CPU 95%。
  - **双 11 零点**：5 秒涌进 3 万次下单 → MySQL p99 飙到 1.4s → 网关 502 → 雪崩。

故事原型：*docs/blog/06-http-cache.md & docs/blog/07-order-async-buffer.md*

两个升级，两条业务承诺

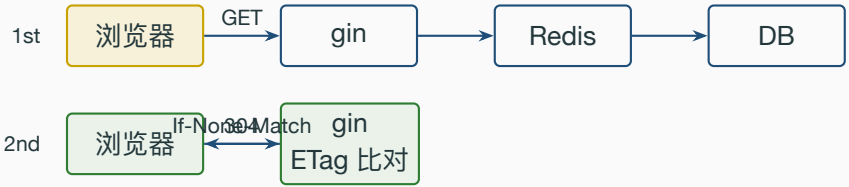


- 两个东西都是”已有功能上加一段中间层”，不动业务语义。
- 共同点：都把”瞬时不可控”转成”可缓冲、可降级”。
- 共同点：都设计了**静默降级**，挂了不影响主链路。

middleware/httpcache.go:47–100；service/order_async.go:119–172

HTTP cache 的业务承诺

第二次请求该走哪条路



- 第一次：全链路，30ms。
- 第二次（同 ETag）：1ms 回 304，body 0 字节。
- 同一个商品被 10 万人刷新 → 后端只算一次 SHA，10 万次 304。

middleware/httpcache.go:78–98 计算 ETag + 命中回 304

业务收益的三笔账

角色	看见	数量级
用户	首屏速度	30ms → <1ms（304 命中）
平台	出网带宽	商品域 1.2 Gbps → 280 Mbps
SRE	DB / ES 回源	QPS 3.2k → 0.8k
财务	CDN 账单	流量 ↓ 60–80%

第一行用户感知最浅，但叠到

GMV 上：” 首屏从 30ms 提到 1ms” 对应业界 +2–3% 转化。

第三、四行才是直接省钱的：DB 砍 4 倍 = 少买 3 台从库。压测基线：*stressTest/REPORT.md §2* 商品详情 *62,226 RPS / p95 3.01ms*

商品价格变化是业务低频事件

- ETag 这套机制的前提：**数据陈旧若干秒，业务可接受**。
- 商品详情真实变更频率：
  - 价格：运营按天 / 大促按小时调，60s 陈旧没人抱怨。
  - 库存：分钟级波动，但库存详情前端只显示” 有货 / 无货”。
  - 文案 / 图：编辑后秒级生效不必要，5 分钟可接受。
- 所以 TTL 选 60s 是**业务 + 运营共识**的结果，不是开发拍的。
- 真正秒级变化的字段（库存数）不走 GET 详情，前端单开” 是否有货” 接口。

routes/router.go:40 product/show 60s；:39 product/list 30s；:43 category 300s；:44 carousels 300s

什么能挂、什么不能挂

分类	示例	能否挂
公开 GET (无登录态)	product/show, category/list, carousels	✓
公开 GET (列表分页)	product/list	✓(短 TTL) **不挂私**
私有 GET (因人而异)	favorites/list, orders/list, carts/list	×
鉴权 GET (用户信息)	user/show_info	×
写接口	POST anything	×

有 GET 是 P0 安全红线：CDN / 浏览器 cache 不会区分用户，A 的购物车回给 B = 安全事故。哪怕 `Cache-Control: private`，公用电脑（机场、网吧）也会出问题。

routes/router.go:38–44 公开 *GET* 挂载点；*:46–47 authed* 组天然不挂

TTL 怎么订：业务侧的四档

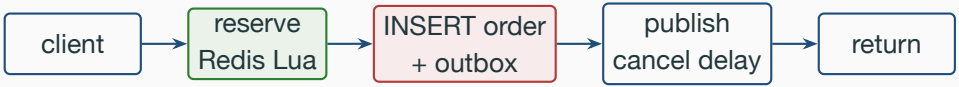
30s	60s	300s	300s
product/list	product/show	category	carousels

- **TTL = 业务可容忍的” 用户看到老数据的最大时长”。**
- 列表 30s：翻页快照本来就有损，30s 足够。
- 详情 60s：调价后 1 分钟同步全网，运营接受。
- 分类 / 轮播 300s：天级变动 + 留余量做灰度发布。
- **TTL 越长越省钱，越短越新鲜**，两个角度业务侧自己拉锯。

routes/router.go:39–44 四个 GET 的 TTL 选型

异步下单的业务承诺

同步下单：双 11 零点的真实链路

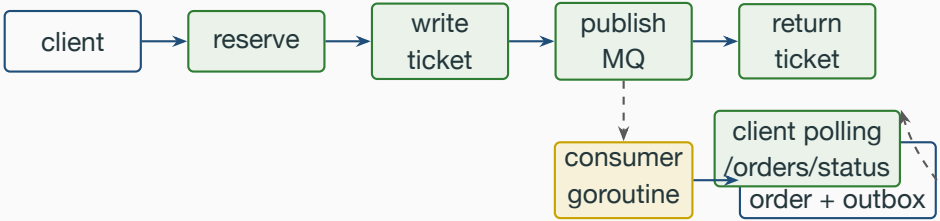


瓶颈：磁盘 fsync + 事务锁

- 平时：30–50ms 一笔，舒舒服服。
- 零点：5 秒 3 万笔 → INSERT 那一段 p99 飙到 1.4s。
- 网关 502 → 前端重试 → 雪崩。
- Redis Lua 那段（reserve）顶得住 5w QPS，是 DB 那段顶不住。

service/order_async.go:118–172 异步路径与同步路径对比

异步下单：把 DB 那段甩到后台



- 同步段只剩：reserve + 写 ticket + publish → < 10ms。
- DB 那段交给 consumer，prefetch=32 自然限速。
- 前端用 ticket 轮询 `/orders/status` 拿结果。

repository/rabbitmq/order_async.go:50–71 ConsumeOrderAsync + Qos(32)；

service/order_consumer.go:51–102 落 DB

业务侧的”已受理”概念

- 异步化之后，HTTP 200 不再意味着”订单已落库”。
- 它的语义变成：**下单意图已被系统接受 + 库存已预占**。
- 这两件事就够了：用户不会被超卖、不会被重复扣款。
- DB 落库是**最终一致**的，秒级延迟用户感知不到。

> 业务承诺从”已下单”降级到”已受理”——但用户感知反而更好（不转圈）。

service/order_async.go:23–29 OrderTicketStatus 三态 pending/ok/failed

两条链路的用户感知时间线

- 用户其实不在乎”几毫秒”，在乎**有没有反馈**。
- 异步链路一上来就给 ticket，前端可以**立刻**显示”下单中”。
- 超过 3 秒切”长等待”文案 + 一键去订单列表，转化损失最小化。

docs/blog/07-order-async-buffer.md 轮询节奏建议 *500ms / 1s / 2s / 3s*

ticket 过期 / 丢失的兜底

- Redis ticket TTL = **1 小时**（service/order_async.go:24）。
- 用户拿到 ticket 就关了浏览器：consumer 落 DB 成功 → ticket 写 ok → 静静躺在 Redis 1 小时。
- 用户 30 分钟后回来：**ticket 还在**，能查到订单号；或者直接去订单列表能看见。
- TTL 到期 + 用户找客服：” 我下了单但没拿到订单号”：
  - 客服按 **idempotency-key + userID + 时间窗**反查 order 表。
  - idempotency 中间件保证同 key 只下一单（业务码 60002），可以追溯。

service/order_async.go:24 OrderTicketTTL = time.Hour；routes/router.go:80 enqueue 也挂 Idempotency

压测数字对照

基线 vs 两个升级（实测 + 推算）

链路	RPS	p95	来源
ping 基线	64,254	3.51ms	实测
商品详情（不挂 cache）	62,226	3.01ms	实测
商品详情（304 命中）	≈ ping 基线	<1ms	推算
同步 /orders/create	50,319	2.33ms	实测
异步 /orders/enqueue	≈ ping 基线	<10ms	推算

- ETag 304 不写 body，理论 RPS 逼近裸 gin 基线。
- enqueue 的同步段只剩 reserve + 写 Redis + publish，RTT 量级 ms。
- **两者都把瓶颈从”业务逻辑”挪到了”网络 + 中间件链”**，DB 不再是上限。

stressTest/REPORT.md § 各链路压测表 1–7 行

接口 SLO 的前后对照

接口	原 p95	优化后 p95	业务承诺
product/show	3.01ms	<1ms (304)	首屏快 + 带宽省
product/list	ms 级	<1ms (304)	翻列表丝滑
orders/create	2–50ms	<10ms (enqueue)	零点不丢单

对照 deck 10 的

SLO 表：P0 (orders/create) 承诺 99.95% / p99 < 500ms。
异步化后 p99 从”DB 不可控” 变成”MQ + Redis 可控”，error budget 烧得慢得多。

stressTest/REPORT.md § 链路压测表；docs/slides/10-business-traffic-governance.tex SLO 表

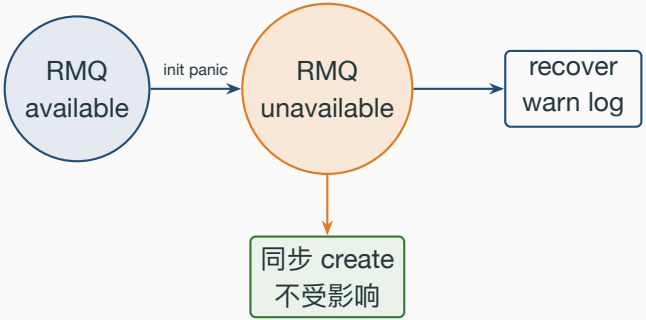
两个升级各自的回滚预案

升级	故障形态	回滚动作
HTTP cache	用户看到老价格客诉	TTL 改 0，只走协商缓存
HTTP cache	304 比对耗 CPU	整条 middleware 摘掉
异步下单	consumer 挂	监控积压告警 + 切回同步
异步下单	ticket 永久 pending	TTL 1h 自然清理 + 客服反查

- HTTP cache 摘除是**单行 router 改动**，秒级回滚。
- 异步下单切回是**前端按钮切流量**，新接口 enqueue 失败时已经 fallback 同步 create。
- 两者都**不破坏旧接口语义**，回滚不需要数据迁移。

routes/router.go:39–44, 78–81 新旧接口共存

静默降级的代码模式



- `tryInitRabbitMQ` 用 `defer recover` 套住 panic，warn 日志后继续。
- Consumer 没起来：`/orders/enqueue` publish 阶段报错，前端 fallback 同步 `/orders/create`。
- ES / Web3 / Milvus 用同一套路 → **基础设施挂一个不连坐。**

cmd/main.go:54–62 tryInitRabbitMQ; :64–73 tryInitES; :86–97 tryInitMilvus

产品 / 运营 / 客服 / SRE 关心什么

角色	关心	HTTP cache 看	异步下单看
产品	转化率	首屏 LCP	ticket 落单成功率
运营	大促节奏	CDN 命中率	enqueue p95
客服	投诉单	” 显示老价格”	” 下单中无响应”
SRE	容量	DB / ES 回源 QPS	consumer 积压

四个角色看同一个升级的

不同切面：
产品看的是收益，运营 / SRE 看的是水位，客服看的是异常。
任何一个面” 看不见” → 这个升级就缺一只眼睛。对照 *deck 10* 的” 各角色关心的指标 + *error budget*”

运营关心：CDN 命中率怎么监控

- gomall 自身不接 CDN（单机部署），但 ETag 收益直接体现在：
  - **后端 304 比例**：访问日志 status=304 占 GET 总数。
  - **出网带宽**：商品域同时段 Gbps，预期降 60–80%。
  - **Redis / DB 读 QPS**：商品域 read replica 应当掉一个数量级。
- 大促前一周抓 baseline，大促当天看曲线。
- 命中率 <50% → 检查前端是否丢了 If-None-Match 头 / TTL 是否过短。

middleware/httpcache.go:81–88 304 响应路径，可由 access log 区分

SRE 关心：consumer 积压告警怎么设

- RMQ 队列 `order.create.async.queue` 深度 = **核心 SLI**。
- 三档告警：
  - P3：深度 > 1000 持续 1min → 钉钉提醒
  - P1：深度 > 10000 持续 30s → 电话值班
  - P0：深度 > 100000 或 consumer 全停 → 自动切回同步路径
- 自动切回的开关：网关侧把 `/orders/enqueue` 流量改路由到 `/orders/create`。
- 这意味着前端拿到的不再是 ticket 而是订单号，UI 侧需要兼容两种响应。

repository/rabbitmq/order_async.go:11–15 队列名 OrderAsyncQueue

回到业务：性能即业务

把这一份 deck 浓缩成一句话

- HTTP cache 不是” 加层缓存”，是**让浏览器替后端挡 80% 流量**。
- 异步下单不是” 换 MQ”，是**把” 用户等” 换成” 系统排队”**。
- 两者都不动业务语义，都**加在已有接口的旁边**，挂了能秒切回。
- 业务方拿到的是两条具体承诺：**运营推爆款不打挂详情、双 11 零点不丢单**。

性能升级的衡量标准不是” 快了多少毫秒”，是” 挽救了多少 GMV、省了多少带宽、少接了多少客诉”。

业务 Q&A（上）

Q1：ETag vs Last-Modified vs Cache-Control，业务侧怎么选？

A：`Cache-Control: max-age` 是**强缓存**，TTL 内根本不发请求，省 RTT。`ETag` / `Last-Modified` 是**协商缓存**，仍发请求但服务端可以 304 省 body。Last-Modified 精度到秒，对秒级变更（商品改价）会误判；ETag 走 body 摘要，精确。gomall 选 Cache-Control + ETag 双管齐下：60s 内零请求，过期发协商。

Q2：cache 击穿时的 SETNX 回源锁怎么和 ETag 配合？

A：两者治理的是不同层。SETNX 锁挡的是**服务端 Redis 失效后多并发回源 DB**；ETag 挡的是**客户端层重复请求**。一个用户 If-None-Match 命中 304 → 根本不进 Redis 那一层 → 也不会触发回源锁。两个机制独立工作，互不影响。

Q3：ticket TTL 1h，用户 2 小时后回来怎么办？

A：ticket 没了，但订单大概率已经在 `orders/list` 里。极端场景（consumer 失败 + ticket 过期）：客服按 idempotency-key + userID + 时间窗反查。TTL 选 1h 是平衡 Redis 内存（千万级 ticket = GB 级）与” 用户回看耐心” 的结果。

业务 Q&A（下）

Q4：异步下单跟秒杀场景怎么合纳?

A：秒杀走 `/skill_product/skill`，挂滑动窗口 3/s/用户，主要拦黑产。普通双 11 / 限时活动走 `/orders/enqueue`，主要削 DB 峰。两者可以叠加：秒杀通过后转 enqueue 落库，DB 完全不见瞬时峰。

Q5：双 11 之外平时是否启用异步?

A：可以。平时 `/orders/enqueue` p95 < 10ms，比同步 30–50ms 还快。但有两个代价：(a) 前端要多写一段轮询；(b) ticket 给运维多一类需要监控的对象。小公司建议平时同步、大促异步，前端按 feature flag 切。

Q6：CDN 命中率怎么监控?

A：(a) 后端访问日志按 status=304 分组，看 304 / (200+304) 比例；(b) 接 CDN 厂商的话直接看 CDN 控制台命中率；(c) 看 DB / ES read QPS 变化幅度，副指标但很可靠。gomall 单机部署只能用 (a) + (c)。

代码位置一览

HTTP cache 中间件 *middleware/httpcache.go:47–100*
弱 ETag 算法 *middleware/httpcache.go:103–106 SHA-256 前 16 字节*
公开 GET 挂载点 *routes/router.go:39–44 product/list, product/show, category, carousels*
异步下单入口 *routes/router.go:80 /orders/enqueue + Idempotency*
ticket 状态查询 *routes/router.go:81 /orders/status*
OrderEnqueue 主流程 *service/order_async.go:119–172*
ticket 三态定义 *service/order_async.go:23–29*
consumer 落 DB + outbox *service/order_consumer.go:51–102*
RMQ 拓扑 *repository/rabbitmq/order_async.go:11–32*
publish / consume *repository/rabbitmq/order_async.go:35–72*
消费者启动 + 静默降级 *initialize/order_async.go:13–29*
main 静默降级套路 *cmd/main.go:54–97 四个 tryInit**

配套博客：docs/blog/06-http-cache.md & docs/blog/07-order-async-